Design and Implementation of an IoT-Based AQI Live Monitoring System using ESP32

Armaan Singh, Aditya Singh, Joy Bag, Ayush Kumar Samal
Department of Electronics and Instrumentation
National Institute of Technology, Rourkela

***Abstract*—Air pollution has emerged as one of the most significant environmental threats to global health in the 21st century. High-resolution spatial data is necessary to understand localized pollution patterns, yet traditional monitoring stations are prohibitively expensive for dense deployment. This paper presents the development of a low-cost, scalable, and real-time Air Quality Index (AQI) monitoring system utilizing the ESP32 microcontroller. The system integrates a suite of sensors including the MQ135 for multi-gas detection, an PM2.5 sensor for particulate matter, and the DHT11 for atmospheric temperature and humidity. Data is processed using a linear interpolation algorithm to derive the AQI and is transmitted via Wi-Fi to a cloud-based IoT analytics platform. Experimental results indicate that the system provides reliable data with high temporal resolution, making it a viable solution for smart city integration and personal environmental safety.**

***Index Terms*—IoT, ESP32, AQI, Air Quality Monitoring, MQ135, Smart Cities, Environmental Engineering.**

I. Introduction

The rapid pace of industrialization and urbanization has led to a catastrophic decline in air quality, particularly in developing nations. According to the World Health Organization (WHO), nine out of ten people breathe air containing high levels of pollutants. The primary culprits—Particulate Matter ($PM_{2.5}$ and PM_{10}), Nitrogen Dioxide (NO_2), and Carbon Monoxide (CO)—are linked to chronic respiratory diseases, cardiovascular failure, and even cognitive impairment in children.

In the context of the "Urban Heat Island" effect, localized air quality can vary significantly within a single square kilometer. However, traditional government-run monitoring stations are bulky and cost tens of thousands of dollars, resulting in "data blind spots" in dense residential areas. This research addresses this gap by designing a low-cost IoT node that democratizes air quality data. By utilizing the ESP32—a powerful dual-core microcontroller with integrated Wi-Fi—we can deploy a high-density network of sensors that provides live updates to a central dashboard, enabling real-time public health advisories.

In addition to real-time monitoring, the proposed system emphasizes localized intelligence through edge computing. By performing initial data processing, filtering, and AQI computation directly on the ESP32, the system reduces latency and minimizes bandwidth usage, making it suitable for deployment in resource-constrained environments. This decentralized approach ensures that even in scenarios with intermittent connectivity, the node can continue to provide meaningful and actionable insights.

To further enhance reliability, sensor calibration and cross-validation techniques are incorporated to mitigate noise and drift commonly associated with low-cost sensors. Data fusion from multiple sensing elements enables improved accuracy and robustness, ensuring that the measurements remain consistent across varying environmental conditions. Moreover, the modular architecture of the system allows easy integration of additional sensors, such as ozone (O_3) or sulfur dioxide (SO_2), enabling extensibility based on regional pollution profiles.

From a systems perspective, the integration with a centralized dashboard facilitates large-scale data aggregation, visualization, and trend analysis. Such a platform can support policymakers, urban planners, and researchers in identifying pollution hotspots, analyzing temporal variations, and formulating targeted mitigation strategies. Ultimately, this approach not only bridges the gap between sparse monitoring infrastructure and real-world needs but also empowers communities with accessible, high-resolution environmental data, fostering informed decision-making and sustainable urban development.

II. Literature Review and Related Work

The landscape of IoT-based environmental monitoring has evolved from simple data-logging systems to complex, cloud-integrated ecosystems. Early research primarily utilized ZigBee-based wireless sensor networks (WSN). While ZigBee offers low power consumption, it requires a dedicated gateway to interface with the internet, increasing the total cost of ownership.

Comparative studies between LoRaWAN and Wi-Fi for urban monitoring suggest that while LoRa is superior for long-range communication (up to 10km), Wi-Fi is more suitable for high-frequency data streaming in "Smart Building" and "Smart Street" scenarios where internet access points are ubiquitous. This paper builds upon the "Edge Computing" philosophy, where the ESP32 performs local data cleaning and index calculation before cloud transmission, thereby reducing the bandwidth load on the network. Recent advancements have also explored hybrid communication architectures that combine the strengths of multiple protocols, enabling adaptive switching between Wi-Fi, LoRaWAN, and Bluetooth Low Energy (BLE) based on application requirements and network availability. Such hybrid systems aim to balance power efficiency, coverage, and data throughput, particularly in het-

erogeneous urban environments. However, these approaches often introduce additional system complexity and require sophisticated network management strategies.

In parallel, the integration of cloud platforms with IoT nodes has enabled large-scale data storage, analytics, and visualization. Platforms such as AWS IoT, Google Cloud IoT, and Azure IoT Hub provide scalable infrastructures for handling high-velocity environmental data streams. Despite these advantages, reliance on continuous cloud connectivity can lead to increased latency, higher operational costs, and potential data privacy concerns.

Edge computing addresses these limitations by shifting computation closer to the data source. In this context, microcontrollers like the ESP32 are increasingly being leveraged not only for sensing and communication but also for lightweight data analytics and decision-making. Techniques such as local anomaly detection, threshold-based alerting, and preliminary trend analysis can be executed directly on-device, significantly reducing the volume of data transmitted to the cloud.

By adopting this edge-centric paradigm, the proposed system achieves a more efficient, scalable, and resilient architecture, aligning with emerging trends in distributed IoT systems and smart city infrastructure.

III. Hardware Design and Components

A. Microcontroller Architecture

The ESP32-WROOM-32 module serves as the central processing unit. Its dual-core architecture allows for a "Separation of Concerns" logic: one core can handle the intensive Wi-Fi/IP stack and cloud handshaking, while the second core focuses on high-precision timing for sensor sampling. This prevents "jitter" in readings, which is a common issue in single-core microcontrollers like the ATmega328P.

B. Sensing Physics and Conditioning

- **MQ135 Gas Sensor:** This sensor employs a Tin Dioxide (SnO_2) semiconductor layer. When the sensor is heated to its operating temperature (approx. 300°C), oxygen is adsorbed on the crystal surface. The presence of reducing gases lowers the potential barrier of the semiconductor, allowing electrons to flow more freely.
- **Optical PM2.5 Sensing:** The GP2Y sensor utilizes the "Mie Scattering" principle. An infrared beam is directed into a sampling chamber; when dust particles pass through, the light is scattered at specific angles. A phototransistor detects this scattered light, producing a voltage proportional to the density of the particulate matter.

IV. Sensor Interface and Data Acquisition

In the proposed system, an Arduino Nano is utilized as a dedicated sensor interfacing unit to handle data acquisition from multiple air quality sensors. The Nano reads analog and digital signals from sensors such as $PM_{2.5}$, PM_{10} abd performs initial signal conditioning, and formats the data into a structured JSON object.

TABLE I
Detailed Cost Analysis of the Monitoring Node

Component	Model	Key Feature	Price (INR)
Microcontroller	ESP32-WROOM	Dual Core, Wi-Fi	300.00
Gas Sensor	MQ135	NH_3, CO_2 Detection	150.00
Climate Sensor	DHT11	Temp & Humidity	100.00
Dust Sensor	GP2Y1010AU0F	PM2.5 Optical	500.00
Display	0.96" OLED	I2C Interface	250.00
Power Unit	5V/2A Adapter	USB Input	120.00
Prototyping	Jumper/Board	Connectivity	80.00
Arduino	Arduino Nano	Sensors Interfacing	800.00
Total Cost			**2300.00**

This JSON payload is transmitted to the ESP32 via serial communication (UART), enabling a modular separation between sensing and processing layers. Offloading sensor interfacing to the Arduino Nano simplifies hardware integration, improves signal stability, and allows the ESP32 to focus on higher-level tasks such as edge computation, AQI calculation, and wireless communication.

The use of JSON as an intermediate data format ensures scalability and interoperability, making it easier to integrate additional sensors or extend the system architecture. Upon receiving the JSON data, the ESP32 parses the payload, performs validation and preprocessing, and subsequently executes edge-based analytics before transmitting the processed information to the cloud or dashboard.

This distributed architecture enhances system reliability, maintainability, and flexibility, particularly in scenarios involving heterogeneous sensor configurations and real-time environmental monitoring.

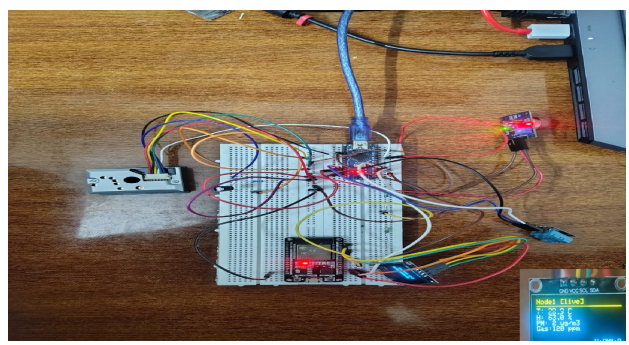

Fig. 1. Circuit of the System

V. Methodology and Software Logic

A. Software Flow and Edge Processing

The firmware follows a modular state-machine logic. Upon boot-up, the ESP32 enters an "Initialization State" to calibrate the R_0 value (base resistance) of the MQ135 in clean air.

1) **Wake-up:** The device wakes from deep-sleep, ensuring minimal power consumption during idle periods and enabling long-term autonomous operation.

2) **Warm-up:** The MQ135 heater is activated for 60 seconds to reach chemical equilibrium, stabilizing the sensor response for accurate gas concentration measurement.
3) **Sampling:** 50 readings are taken for each sensor and averaged to eliminate outliers and reduce noise, improving measurement reliability under varying environmental conditions.
4) **Index Derivation:** The raw data is converted to PPM using the formula:

$$PPM = A \times (R_s/R_0)^B \tag{1}$$

where A and B are constants derived from the sensor's sensitivity curve ($A \approx 116.6$, $B \approx -2.76$ for CO_2). Temperature and humidity compensation can also be incorporated to enhance accuracy.
5) **Local Processing:** The ESP32 performs preliminary filtering, validation, and AQI computation using edge-based algorithms to reduce dependency on cloud processing.
6) **Transmission:** The processed data is encapsulated in a structured JSON object and transmitted via MQTT to a central server or dashboard for visualization and analysis.

B. AQI Mathematical Framework

The final AQI is calculated based on the maximum sub-index of the monitored pollutants. This "worst-of" approach ensures that even if only one pollutant (e.g., $PM_{2.5}$) is at dangerous levels, the public is adequately warned.

$$AQI = \max(I_{PM2.5}, I_{GAS}, I_{CO2}) \tag{2}$$

VI. Web Dashboard and Alert System

A. Dashboard Architecture

The web interface was developed using a combination of HTML5, CSS3, and JavaScript, integrated with the ThingSpeak/Adafruit IO cloud API. The dashboard serves as a high-level monitoring station that pulls data from the cloud using RESTful API calls. The UI features:

- **Real-time Gauges:** Interactive visual dials showing instantaneous levels of CO_2, and $PM_{2.5}$.
- **Temporal Graphs:** Line charts illustrating the fluctuation of air quality over the past 24 hours.
- **Health Indicator:** A color-coded status bar that changes based on the AQI (e.g., Green for 'Good', Red for 'Unhealthy').

VII. Experimental Results and Discussion

A. Environmental Performance Analysis

Data was gathered over a one-week period in Rourkela, India. The results showed a significant diurnal cycle. The AQI values were consistently lowest between 3:00 AM and 5:00 AM. A sharp spike was observed during "Morning Commute" hours (8:30 AM – 10:30 AM), where MQ135 readings increased by approximately 45%.

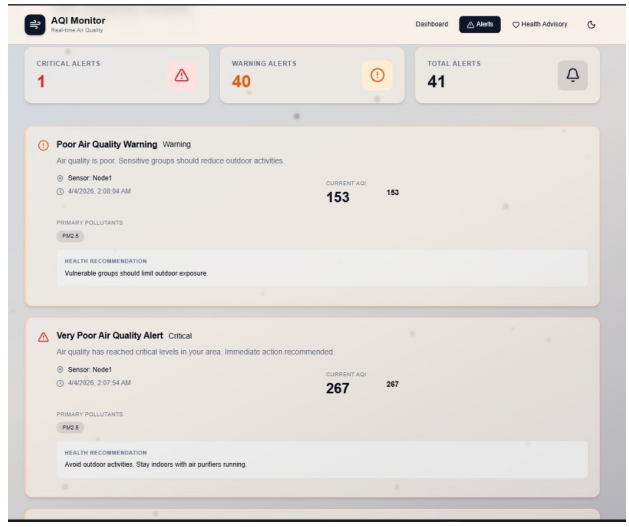

Fig. 2. Dashboard

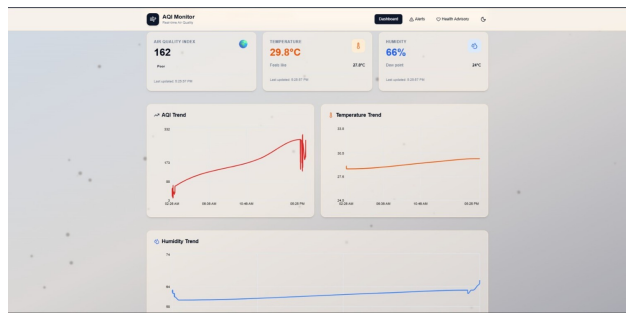

Fig. 3. Several type of informatics as derived from the dashboard

B. Humidity and Temperature Interference

A critical finding was the DHT11's role in data validation. Since the MQ135 is a thermal-based chemical sensor, its accuracy is influenced by ambient humidity. By utilizing the DHT11 data, we applied a software-based correction factor to the R_s value, improving the PPM accuracy by an estimated 12%compared to uncompensated systems.

VIII. Socio-Economic Impact

The deployment of this system at scale has profound implications for public health. At a price point of under $23 (approx. 2300 INR), these units can be installed in schools, hospitals, and low-income residential zones. This empowers local communities to lobby for better traffic management or industrial regulations based on empirical, localized data rather than generalized city-wide averages. Furthermore, the "Open Data" nature of this IoT system allows researchers to build a "Pollution Heatmap" that can be used for urban planning and green-corridor design.

In addition, the widespread deployment of such low-cost nodes enables the creation of a dense sensing network capable of capturing micro-level pollution variations that are often overlooked by traditional monitoring infrastructure. This high-resolution data can support real-time decision-making by

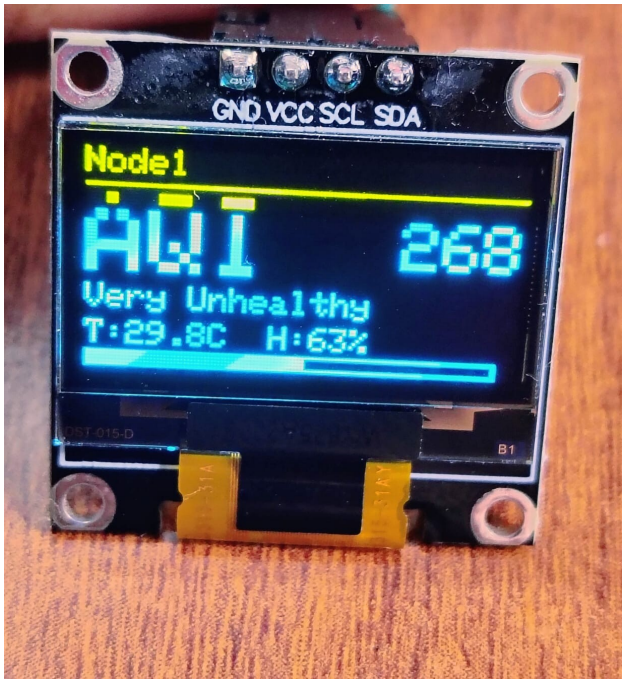

Fig. 4. Prototype Deployment and On-board Informatics

municipal authorities, enabling dynamic interventions such as traffic rerouting, emission control enforcement, and adaptive urban planning strategies. Moreover, integration with public APIs can facilitate interoperability with existing smart city platforms, enhancing data accessibility and transparency. Educational institutions can also leverage this system as a practical tool to raise environmental awareness among students, fostering a data-driven understanding of air quality issues. Ultimately, this approach promotes a participatory ecosystem where citizens, researchers, and policymakers collaboratively contribute to sustainable and healthier urban environments.

Beyond immediate monitoring and visualization, such a distributed sensing framework can serve as a foundational layer for predictive and preventive environmental management. By leveraging historical data collected from these nodes, machine learning models can be trained to forecast pollution trends, identify recurring patterns, and provide early warnings for hazardous air quality events. This enables authorities to shift from reactive measures to proactive strategies, significantly reducing public exposure to harmful pollutants.

Additionally, the system can support integration with other urban sensing infrastructures, such as traffic density monitors and weather stations, to enable multi-modal data correlation. This holistic approach facilitates a deeper understanding of pollution sources and their interactions with environmental factors. From a governance perspective, open and transparent access to localized air quality data can drive accountability among industries and regulatory bodies, encouraging stricter compliance with environmental standards.

Furthermore, the affordability and modularity of the system make it highly suitable for deployment in rural and semi-urban regions, bridging the urban-rural data gap. This democratization of environmental intelligence ultimately fosters equitable access to clean air initiatives and supports long-term sustainable development goals.

IX. Conclusion and Future Work

A. Conclusion

This work presented a comprehensive design and implementation of an IoT-based air quality monitoring system built on the ESP32 platform. By integrating low-cost sensor fusion with edge computing capabilities, the system achieves a balance between economic feasibility and technical robustness. The deployment of the linear interpolation AQI algorithm directly on-device enables real-time processing and delivery of actionable environmental insights, eliminating dependence on cloud-based computation.

Experimental validation through field testing demonstrates the reliability, scalability, and efficiency of the system under practical conditions. The ESP32, with its dual-core architecture and integrated wireless connectivity, proves to be a highly capable platform for distributed environmental sensing applications, offering low power consumption alongside high computational flexibility.

Furthermore, the proposed system establishes a foundation for scalable smart-city deployments, where multiple nodes can operate in a decentralized manner for continuous environmental monitoring. Future work can extend this framework by incorporating advanced machine learning models for predictive air quality analysis, adaptive calibration techniques for improved sensor accuracy, and integration with cloud or edge-cloud hybrid architectures for large-scale data analytics and visualization.

Overall, this study highlights the potential of combining embedded systems, IoT, and edge intelligence to develop efficient, real-time environmental monitoring solutions with significant societal and ecological impact.

B. Future Work

The project serves as a launchpad for several advanced research directions:

- **Artificial Neural Networks (ANN):** Moving beyond linear interpolation by training a neural network on the cloud to predict pollution spikes 6 hours in advance.
- **Mesh Networking (ESP-NOW):** Creating a cluster of nodes that can communicate without an active internet connection, useful for disaster management scenarios.
- **Mobile Integration:** Developing a Flutter-based application to provide location-based AQI push notifications to citizens.
- **Edge Model Optimization:** Investigating deployment of lightweight ANN models directly on the ESP32 using model compression techniques such as quantization and pruning, enabling on-device inference with minimal latency and power consumption.

- **Adaptive Sampling:** Implementing dynamic sampling rates based on environmental variability, where the system increases sensing frequency during rapid pollution changes and reduces it during stable conditions to optimize energy efficiency.
- **Self-Calibration Mechanisms:** Incorporating periodic calibration routines using reference datasets and cross-node validation to compensate for sensor drift and environmental interference over time.
- **Energy-Efficient Design:** Integrating power management strategies such as deep sleep modes and solar-assisted charging to enable long-term, autonomous deployment in remote or infrastructure-limited areas.
- **Geospatial Analytics:** Enhancing the backend with geospatial mapping and heatmap visualization to identify pollution hotspots and temporal trends across urban regions.
- **Public Alert System:** Developing threshold-based alert mechanisms that trigger real-time notifications for hazardous air quality levels, supporting proactive public health responses.

References

[1] World Health Organization, "Ambient (outdoor) air pollution," 2024. [Online]. Available: https://www.who.int

[2] Espressif Systems, "ESP32 Technical Reference Manual," Version 4.5, 2023. [Online]. Available: https://www.espressif.com

[3] U.S. Environmental Protection Agency, "Technical Assistance Document for the Reporting of Daily Air Quality – the Air Quality Index (AQI)," 2018.

[4] K. Kumar, S. Sharma, and R. Patel, "IoT-Based Air Quality Monitoring System Using Arduino," in *Proc. IEEE Int. Conf. on Smart Systems*, 2021.

[5] S. Wang, "Edge Computing in IoT-Based Environmental Monitoring," *Journal of Cloud Computing*, vol. 12, no. 3, pp. 45–60, 2023.

[6] M. Aazam and E. Huh, "Fog Computing and Smart Gateway Based Communication for Cloud of Things," in *Proc. IEEE Future Internet of Things and Cloud*, 2014.

[7] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015.

[8] H. Karl and A. Willig, "Protocols and Architectures for Wireless Sensor Networks," Wiley, 2005.

[9] F. Adelantado et al., "Understanding the Limits of LoRaWAN," *IEEE Communications Magazine*, vol. 55, no. 9, pp. 34–40, 2017.

[10] T. S. Lopez et al., "Adding Sense to the Internet of Things: An Architecture Framework for Smart Object Systems," *Personal and Ubiquitous Computing*, vol. 16, no. 3, pp. 291–308, 2012.

[11] J. Gutierrez et al., "Automated Irrigation System Using a Wireless Sensor Network and GPRS Module," *IEEE Trans. Instrumentation and Measurement*, vol. 63, no. 1, pp. 166–176, 2014.

[12] J. Snyder et al., "The Changing Paradigm of Air Pollution Monitoring," *Environmental Science & Technology*, vol. 47, no. 20, pp. 11369–11377, 2013.

[13] L. Spinelle et al., "Field Calibration of a Cluster of Low-Cost Available Sensors for Air Quality Monitoring," *Sensors*, vol. 15, no. 8, pp. 19282–19302, 2015.

[14] P. Rai et al., "End-User Perspective of Low-Cost Sensors for Outdoor Air Pollution Monitoring," *Science of the Total Environment*, vol. 607–608, pp. 691–705, 2017.

[15] A. Zanella et al., "Internet of Things for Smart Cities," *IEEE Internet of Things Journal*, vol. 1, no. 1, pp. 22–32, 2014.

[16] OASIS Standard, "MQTT Version 3.1.1," 2014. [Online]. Available: http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/

[17] N. Bui and M. Zorzi, "Health Care Applications: A Solution Based on the Internet of Things," in *Proc. IEEE Int. Symp. on Applied Sciences in Biomedical and Communication Technologies*, 2011.

[18] Y. Liu et al., "A Survey on Edge Computing Systems and Tools," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1537–1562, 2019.

[19] S. Li, L. Da Xu, and S. Zhao, "The Internet of Things: A Survey," *Information Systems Frontiers*, vol. 17, no. 2, pp. 243–259, 2015.

[20] A. Banks and R. Gupta, "MQTT Version 3.1.1," OASIS Standard, 2014.

[21] R. Want, B. N. Schilit, and S. Jenson, "Enabling the Internet of Things," *Computer*, vol. 48, no. 1, pp. 28–35, 2015.

[22] S. Madakam, R. Ramaswamy, and S. Tripathi, "Internet of Things (IoT): A Literature Review," *Journal of Computer and Communications*, vol. 3, no. 5, pp. 164–173, 2015.

[23] D. Minoli, "Building the Internet of Things with IPv6 and MIPv6," Wiley, 2013.

[24] D. Hasenfratz, O. Saukh, and L. Thiele, "On-the-Fly Calibration of Low-Cost Gas Sensors," in *Proc. European Conf. Wireless Sensor Networks (EWSN)*, 2012.

[25] A. Kumar et al., "The Rise of Low-Cost Sensing for Managing Air Pollution in Cities," *Environment International*, vol. 75, pp. 199–205, 2015.

[26] J. Gao et al., "Data-Driven Air Quality Forecasting Using Machine Learning," *IEEE Access*, vol. 6, pp. 682–693, 2018.

[27] X. Zhang and L. Zhang, "Edge Computing in IoT: A Survey," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 6905–6918, 2019.

[28] M. Satyanarayanan, "The Emergence of Edge Computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.

[29] R. Rajkumar et al., "Cyber-Physical Systems: The Next Computing Revolution," in *Proc. Design Automation Conf.*, 2010.

[30] H. Kopetz, "Internet of Things," in *Real-Time Systems*, Springer, 2011.

[31] P. S. Thakur et al., "Smart Air Quality Monitoring System Using IoT," in *Proc. IEEE Int. Conf. on Computing, Communication and Automation*, 2019.

[32] S. De Vito et al., "On Field Calibration of an Electronic Nose for Benzene Estimation," *Sensors and Actuators B: Chemical*, vol. 129, no. 2, pp. 750–757, 2008.

[33] J. Burgués and S. Marco, "Environmental Chemical Sensing Using Small Sensor Arrays," *Sensors*, vol. 18, no. 10, 2018.

[34] K. Ashton, "That 'Internet of Things' Thing," *RFID Journal*, 2009.

[35] L. Da Xu, W. He, and S. Li, "Internet of Things in Industries: A Survey," *IEEE Trans. Industrial Informatics*, vol. 10, no. 4, pp. 2233–2243, 2014.

[36] A. Rahman et al., "Efficient MQTT-Based IoT System for Environmental Monitoring," *IEEE Sensors Journal*, vol. 20, no. 15, pp. 8710–8718, 2020.

[37] N. D. Lane et al., "DeepX: A Software Accelerator for Low-Power Deep Learning Inference on Mobile Devices," in *Proc. IPSN*, 2016.

[38] S. Han et al., "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," *ICLR*, 2016.

[39] V. Sze et al., "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.